Chapman-Kolmogorov regularizers for sequence evolvers

Ian Holmes

February 22, 2026

Abstract

A few ideas on how to quantify departures from non-Markovian behavior in neural approximators to matrix exponentials for whole-sequence evolutionary models, and some speculation on how this might be operationalized for training.

1 Chapman-Kolmogorov for sequence evolvers

Suppose we have a model for a descendant sequence Y given an ancestral sequence X and an evolutionary time parameter ℓ_1. Denote this model by $M(Y|X,\ell_1) \equiv \Pr(Y|X,\ell_1,M)$.

Consider a two-step evolution $A \xrightarrow{\ell_1} B \xrightarrow{\ell_2} C$ under this model, where a sequence A evolves over time ℓ_1 to another sequence B, which then evolves over time ℓ_2 to a third sequence C.

If we sum out B we get a distribution for C conditioned on A

$$Z(C|A,\ell_1,\ell_2) = \sum_B M(B|A,\ell_1)M(C|B,\ell_2)$$

If the model is Markovian and stationary then it must obey the Chapman-Kolmogorov equation

$$M(C|A,\ell_1+\ell_2) = Z(C|A,\ell_1,\ell_2) \tag{1}$$

for all sequences A, C and times ℓ_1, ℓ_2.

2 Importance sampling

One challenge with using Z is that it requires an infinite sum over intermediates B. However, we may be able to estimate Z using a finite sum.

The function $Z(C|A,\ell_1,\ell_2)$ can be written as an expectation

$$Z(C|A,\ell_1,\ell_2) = \mathbb{E}_{B\sim M(B|A,\ell_1)}\left[M(C|B,\ell_2)\right]$$

and can be viewed as the partition function for an implicit posterior over B

$$\Pr(B|A, C, \ell_1, \ell_2, M) = \frac{M(B|A, \ell_1)M(C|B, \ell_2)}{Z(C|A, \ell_1, \ell_2)}$$

I suggest estimating Z by importance sampling, using a proposal $q(B|A, C, \ell_1, \ell_2)$

$$Z(C|A, \ell_1, \ell_2) = \mathbb{E}_{B\sim q(B|A,C,\ell_1,\ell_2)}\left[\frac{M(B|A, \ell_1)M(C|B, \ell_2)}{q(B|A, C, \ell_1, \ell_2)}\right]$$

For finite sums, the variance of this estimator is smaller if we choose a proposal distribution that is close to M's implicit posterior over B, i.e. $q \simeq \Pr(B|A, C, \ell_1, \ell_2, M)$.

For example, if q is *exactly* that implicit posterior, then we are just taking the expectation of the (constant) Z, and the finite sum would be exact even if we only took one sample. Of course, this is circular reasoning: we do not know the implicit posterior, *because* we don't know Z. Furthermore, even if we could evaluate it for a given B, we might find it hard to generate samples from. However, it illustrates the point that M's posterior for B would be the *ideal* choice for q.

A reasonable choice for q, then, seems to be a posterior distribution for a model whose posterior distribution over intermediaries B we *can* evaluate, since it is Markovian (or almost-Markovian) by construction, and (furthermore) is one we can easily generate samples from—e.g. a model like TKF91, which is exactly solved—its finite-time distributions are expressible as HMMs in closed form—or TKF92, which is almost exactly solved (technically its Markovian state is a partitioned sequence, rather than a sequence *per se*, but in practice this seems not to perturb things too much). If this is a good model, its posterior distribution over intermediate sequences will be not *too* far from M's, and we should be able to approximate Z with a small sample.

3 Python code

The accompanying Python script `sample_intermediates.py` takes as input a set of TKF92 model parameters and a pair of sequences, and generates a list of tuples of the form (B_k, c_k, p_k) where B_k is a sequence, c_k is its multiplicity (the number of times it appeared during sampling, so $\sum_k c_k = N$), and p_k is its posterior probability.

Given a model of the form $M(Y|X, \ell)$ as described in Section 1, the partition function $Z(C|A, \ell_1, \ell_2)$ can be estimated by importance sampling from this set of samples as follows

$$Z(C|A, \ell_1, \ell_2) \simeq \frac{1}{N}\sum_k \frac{M(B_k|A, \ell_1)M(C|B_k, \ell_2)}{c_k p_k} \tag{2}$$

This can then be compared to $M(C|A, \ell_1 + \ell_2)$ as per (1).

The Python script should be reasonably self-documenting. The `--help` option provides usage info. I would recommend using the `--json` option for machine-parseable output.

Section 4 documents the algorithms used by the script. Section 5 outlines how to use this approach to estimate a relative entropy between the expected (Markov-consistent) distribution Z and the computed model distribution M. Section 6 briefly speculates as to how this might be operationalized to train Markov-consistent neural models.

4 TKF92 HMMs and algorithms

4.1 Pair HMM definitions

A Pair HMM $(\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$ has state space Φ, output alphabet Ω, transition matrix $\mathbf{t}$, emission function $\mathbf{e}$, and state type function $\tau : \Phi \to \{\mathtt{S}, \mathtt{M}, \mathtt{I}, \mathtt{D}, \mathtt{N}, \mathtt{E}\}$ (signifying start, match, insert, delete, null, and end). There must be exactly one start state $\phi^{(\mathtt{S})}$ with $\tau(\phi^{(\mathtt{S})}) = \mathtt{S}$, and one end state $\phi^{(\mathtt{E})}$ with $\tau(\phi^{(\mathtt{E})}) = \mathtt{E}$.

The transition matrix entry t_{ij} is the probability that the next state will be j given that the current state is i. It is required that $t_{k,\phi^{(\mathtt{S})}} = t_{\phi^{(\mathtt{E})},k} = 0$ for all $k \in \Phi$.

Emissions occur on entry into states. Let $e_k(\omega, \omega')$ be the probability of emitting ancestral alignment symbol ω and descendant alignment symbol ω' from state k, where $\omega, \omega' \in \Omega \cup \{\epsilon\}$ and ϵ is the empty string (denoting a gap in an alignment context).

For all non-gap $\omega, \omega' \in \Omega$, the emission probabilities must satisfy

$$\begin{aligned}
\tau(k) = \mathtt{M} \quad &\Rightarrow \quad e_k(\omega, \epsilon) = e_k(\epsilon, \omega') = e_k(\epsilon, \epsilon) = 0 \\
\tau(k) = \mathtt{I} \quad &\Rightarrow \quad e_k(\omega, \omega') = e_k(\omega, \epsilon) = e_k(\epsilon, \epsilon) = 0 \\
\tau(k) = \mathtt{D} \quad &\Rightarrow \quad e_k(\omega, \omega') = e_k(\epsilon, \omega') = e_k(\epsilon, \epsilon) = 0 \\
\tau(k) \in \{\mathtt{S}, \mathtt{E}, \mathtt{N}\} \quad &\Rightarrow \quad e_k(\omega, \omega') = e_k(\omega, \epsilon) = e_k(\epsilon, \omega') = 0
\end{aligned}$$

Define the emission indicator functions

$$\begin{aligned}
\Delta_X(k) &= \delta(\tau(k) \in \{\mathtt{M}, \mathtt{D}\}) \\
\Delta_Y(k) &= \delta(\tau(k) \in \{\mathtt{M}, \mathtt{I}\})
\end{aligned}$$

4.2 TKF92 model

The TKF92 model has indel parameters (λ, μ, r) and substitution parameters $(\mathbf{Q}, \pi)$.

In this model, k-residue fragments (where $k > 0$ and $\Pr(k) = r^{k-1}(1-r)$) are inserted and deleted at rates λ, μ. The rate at which any given residue is deleted is just the rate at which its fragment is deleted, μ. The mean number of fragments at equilibrium is $\frac{\lambda/\mu}{1-\lambda/\mu} = \frac{1}{\mu/\lambda - 1}$ and the mean fragment length is $1/(1-r)$, so the mean sequence length at equilibrium is $\frac{1}{(\mu/\lambda-1)(1-r)}$. The precise distribution of sequence lengths at equilibrium is a zero-altered geometric distribution

$$\Pr(n) = \begin{cases} 1 - \frac{\lambda}{\mu} & \text{if } n = 0 \\ \frac{\lambda}{\mu} \nu^{n-1} (1 - \nu) & \text{if } n > 0 \end{cases}$$

where $\nu = r + (1-r)\frac{\lambda}{\mu}$.

The substitution parameters comprise a reversible substitution rate matrix $\mathbf{Q}$ and stationary distribution π which must satisfy $\pi\mathbf{Q} = 0$ (equilibrium) and $\pi_i Q_{ij} = \pi_j Q_{ji}$ (detailed balance).

4.3 TKF91 model

The TKF92 model reduces to the TKF91 model when $r = 0$.

4.4 TKF92 joint transition matrix

$$\texttt{JointPairMatrix}(\lambda, \mu, r, \ell) =$$

$$\left(\begin{array}{c|ccccc}
 & \texttt{S} & \texttt{M} & \texttt{I} & \texttt{D} & \texttt{E} \\
\hline
\texttt{S} & 0 & (1-\beta)\kappa\alpha & \beta & (1-\beta)\kappa(1-\alpha) & (1-\beta)(1-\kappa) \\
\texttt{M} & 0 & r+(1-r)(1-\beta)\kappa\alpha & (1-r)\beta & (1-r)(1-\beta)\kappa(1-\alpha) & (1-r)(1-\beta)(1-\kappa) \\
\texttt{I} & 0 & (1-r)(1-\beta)\kappa\alpha & r+(1-r)\beta & (1-r)(1-\beta)\kappa(1-\alpha) & (1-r)(1-\beta)(1-\kappa) \\
\texttt{D} & 0 & (1-r)(1-\gamma)\kappa\alpha & (1-r)\gamma & r+(1-r)(1-\gamma)\kappa(1-\alpha) & (1-r)(1-\gamma)(1-\kappa) \\
\texttt{E} & 0 & 0 & 0 & 0 & 0
\end{array}\right)$$

$$\begin{aligned}
\alpha(\lambda, \mu, \ell) &= \exp(-\mu\ell) \\
\beta(\lambda, \mu, \ell) &= \frac{\lambda\left(\exp(-\lambda\ell) - \exp(-\mu\ell)\right)}{\mu\exp(-\lambda\ell) - \lambda\exp(-\mu\ell)} \\
\gamma(\lambda, \mu, \ell) &= 1 - \frac{\mu\beta}{\lambda(1-\alpha)} \\
\kappa(\lambda, \mu) &= \frac{\lambda}{\mu}
\end{aligned}$$

4.5 TKF92 joint pair HMM

$$\texttt{JointPairHMM}(\theta, \ell) = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$$

where $\theta = (\lambda, \mu, r, \mathbf{Q}, \pi)$.

This is the full description of the HMM whose transition matrix is given in Section 4.4. Here Ω denotes the sequence alphabet, i.e. the index space of the substitution model $(\mathbf{Q}, \pi)$.

We do not actually use this Pair HMM, or its transition matrix. It is provided only to make sense of the conditional transition matrix presented in Section 4.6 and the associated conditional Pair HMM in Section 4.7.

$$
\begin{aligned}
\Phi &= \{\mathtt{S}, \mathtt{M}, \mathtt{I}, \mathtt{D}, \mathtt{E}\} \\
\mathbf{t} &= \mathtt{JointPairMatrix}(\lambda, \mu, r, \ell) \\
e_{\mathtt{M}}(\omega, \omega') &= \begin{cases} \pi_\omega \exp(\mathbf{Q}\ell)_{\omega\omega'} & \text{if } \omega, \omega' \in \Omega \\ 0 & \text{otherwise} \end{cases} \\
e_{\mathtt{I}}(\omega, \omega') &= \pi_{\omega'} \\
e_{\mathtt{D}}(\omega, \omega') &= \pi_\omega \\
\tau(k) &= k \\
\phi^{(\mathtt{S})} &= \mathtt{S} \\
\phi^{(\mathtt{E})} &= \mathtt{E}
\end{aligned}
$$

4.6 TKF92 conditional transition matrix

The transition matrix for the conditional Pair HMM, which models the probability of a descendant sequence Y conditioned on its ancestor X, is obtained by dividing the X-emitting transitions by the appropriate terms from the zero-altered distribution of ancestral sequence lengths

$\mathtt{ConditionalPairMatrix}(\lambda, \mu, r, \ell) =$

$$
\left(\begin{array}{c|ccccc}
 & \mathtt{S} & \mathtt{M} & \mathtt{I} & \mathtt{D} & \mathtt{E} \\
\hline
\mathtt{S} & 0 & (1-\beta)\alpha & \beta & (1-\beta)(1-\alpha) & 1-\beta \\
\mathtt{M} & 0 & \frac{1}{\nu}(r + (1-r)(1-\beta)\kappa\alpha) & (1-r)\beta & \frac{1}{\nu}(1-r)(1-\beta)\kappa(1-\alpha) & 1-\beta \\
\mathtt{I} & 0 & \frac{1}{\nu}(1-r)(1-\beta)\kappa\alpha & r + (1-r)\beta & \frac{1}{\nu}(1-r)(1-\beta)\kappa(1-\alpha) & 1-\beta \\
\mathtt{D} & 0 & \frac{1}{\nu}(1-r)(1-\gamma)\kappa\alpha & (1-r)\gamma & \frac{1}{\nu}(r + (1-r)(1-\gamma)\kappa(1-\alpha)) & 1-\gamma \\
\mathtt{E} & 0 & 0 & 0 & 0 & 0
\end{array}\right)
$$

4.7 TKF92 conditional pair HMM

$$\mathtt{ConditionalPairHMM}(\theta, \ell) = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$$

where $\theta = (\lambda, \mu, r, \mathbf{Q}, \pi)$.

This is the same as the joint pair HMM (Section 4.5), except the transition matrix is as given in Section 4.6 and the emission probabilities omit the π_ω term for ancestral residues.

Since it's conditionally normalized, it is technically a finite-state transducer rather than a Pair HMM.

$$
\begin{aligned}
\Phi &= \{\mathtt{S}, \mathtt{M}, \mathtt{I}, \mathtt{D}, \mathtt{E}\} \\
\mathbf{t} &= \mathtt{ConditionalPairMatrix}(\lambda, \mu, r, \ell) \\
e_{\mathtt{M}}(\omega, \omega') &= \begin{cases} \exp(\mathbf{Q}\ell)_{\omega\omega'} & \text{if } \omega, \omega' \in \Omega \\ 0 & \text{otherwise} \end{cases} \\
e_{\mathtt{I}}(\omega, \omega') &= \pi_{\omega'} \\
e_{\mathtt{D}}(\omega, \omega') &= 1 \\
\tau(k) &= k \\
\phi^{(\mathtt{S})} &= \mathtt{S} \\
\phi^{(\mathtt{E})} &= \mathtt{E}
\end{aligned}
$$

4.8 Triad HMM

$$\mathtt{TriadHMM}(\theta, \ell_1, \ell_2) = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$$

where $\theta = (\lambda, \mu, r, \mathbf{Q}, \pi)$.

This is the HMM for the conditional distribution $\Pr(B, C|A, \theta, \ell_1, \ell_2)$, reflecting two generations of evolutionary drift: $A \xrightarrow{\ell_1} B \xrightarrow{\ell_2} C$. Technically it is a three-sequence HMM, but we will treat B as unobserved. As such, we represent it as a conditional Pair HMM (or, more precisely, a finite-state transducer) that inputs A and outputs C.

Each state can be considered a tuple comprising two states from the conditional pair HMM of Section 4.7: one for $A \xrightarrow{\ell_1} B$ and one for $B \xrightarrow{\ell_2} C$. The upper/lower case symbols in the state name indicate which component(s) change on transition into the state. For example, transitions into `MD` change both components, but transitions into `iI` only change the second component. (The cases are introduced to make the states and transitions more comprehensible; they do not have any other effect. The transition rules can be stated without reference to cases, and all state names remain unique even if they are all converted to the same case.)

The transducer update rules are designed to keep the first component's token-emitting transitions synchronized with the second component's token-absorbing transitions:

- When the first component is updated, the second component can only update if the first component enters a token-outputting `M`, `I`, or `E` state. In the former two cases (first component goes to `M` or `I`) the second component must go to a `M` or `D` state; in the latter case (first component goes to `E`) the second component must also go to `E`. (The latter case can be understood as the first component passing an unobserved end-of-sequence token to the second component.)
- If the first component transitions to a `D` state, the second component receives no input token, and cannot be updated.
- If the first component is *not* updated, the second component can only transition to an `I` state, which does not require an input token.
- If a component does not update, its state character is lower-cased. (Again, this has no effect on the semantics, except to indicate that the component was quiescent during the last transition.)

Thus, the transition matrix has the general form

$$t_{ik,jl} = \begin{cases} 0 & \text{if } jl = \texttt{SS} \text{ or } ik = \texttt{EE} \\ z_{kl} & \text{if } i \neq \texttt{D},\ l = \texttt{I}, \text{ and } j = \text{toLowerCase}(i) \\ y_{ij} z_{kl} & \text{if } j \in \{\texttt{M}, \texttt{I}\} \text{ and } l \in \{\texttt{M}, \texttt{D}\} \\ y_{ij} & \text{if } j = \texttt{D} \text{ and } l = \text{toLowerCase}(k) \\ y_{ij} z_{kl} & \text{if } jl = \texttt{EE} \\ 0 & \text{otherwise} \end{cases}$$

In detail,

$$\mathbf{t} = \left(\begin{array}{c|ccccccccccccc} & \texttt{SS} & \texttt{sI} & \texttt{MM} & \texttt{mI} & \texttt{MD} & \texttt{IM} & \texttt{iI} & \texttt{ID} & \texttt{Ds} & \texttt{Dm} & \texttt{Di} & \texttt{Dd} & \texttt{EE} \\ \hline \texttt{SS} & 0 & z_{\texttt{SI}} & y_{\texttt{SM}} z_{\texttt{SM}} & 0 & y_{\texttt{SM}} z_{\texttt{SD}} & y_{\texttt{SI}} z_{\texttt{SM}} & 0 & y_{\texttt{SI}} z_{\texttt{SD}} & y_{\texttt{SD}} & 0 & 0 & 0 & y_{\texttt{SE}} z_{\texttt{SE}} \\ \texttt{sI} & 0 & z_{\texttt{II}} & y_{\texttt{SM}} z_{\texttt{IM}} & 0 & y_{\texttt{SM}} z_{\texttt{ID}} & y_{\texttt{SI}} z_{\texttt{IM}} & 0 & y_{\texttt{SI}} z_{\texttt{ID}} & 0 & 0 & y_{\texttt{SD}} & 0 & y_{\texttt{SE}} z_{\texttt{IE}} \\ \texttt{MM} & 0 & 0 & y_{\texttt{MM}} z_{\texttt{MM}} & z_{\texttt{MI}} & y_{\texttt{MM}} z_{\texttt{MD}} & y_{\texttt{MI}} z_{\texttt{MM}} & 0 & y_{\texttt{MI}} z_{\texttt{MD}} & 0 & y_{\texttt{MD}} & 0 & 0 & y_{\texttt{ME}} z_{\texttt{ME}} \\ \texttt{mI} & 0 & 0 & y_{\texttt{MM}} z_{\texttt{IM}} & z_{\texttt{II}} & y_{\texttt{MM}} z_{\texttt{ID}} & y_{\texttt{MI}} z_{\texttt{IM}} & 0 & y_{\texttt{MI}} z_{\texttt{ID}} & 0 & 0 & y_{\texttt{MD}} & 0 & y_{\texttt{ME}} z_{\texttt{IE}} \\ \texttt{MD} & 0 & 0 & y_{\texttt{MM}} z_{\texttt{DM}} & z_{\texttt{DI}} & y_{\texttt{MM}} z_{\texttt{DD}} & y_{\texttt{MI}} z_{\texttt{DM}} & 0 & y_{\texttt{MI}} z_{\texttt{DD}} & 0 & 0 & 0 & y_{\texttt{MD}} & y_{\texttt{ME}} z_{\texttt{DE}} \\ \texttt{IM} & 0 & 0 & y_{\texttt{IM}} z_{\texttt{MM}} & 0 & y_{\texttt{IM}} z_{\texttt{MD}} & y_{\texttt{II}} z_{\texttt{MM}} & z_{\texttt{MI}} & y_{\texttt{II}} z_{\texttt{MD}} & 0 & y_{\texttt{ID}} & 0 & 0 & y_{\texttt{IE}} z_{\texttt{ME}} \\ \texttt{iI} & 0 & 0 & y_{\texttt{IM}} z_{\texttt{IM}} & 0 & y_{\texttt{IM}} z_{\texttt{ID}} & y_{\texttt{II}} z_{\texttt{IM}} & z_{\texttt{II}} & y_{\texttt{II}} z_{\texttt{ID}} & 0 & 0 & y_{\texttt{ID}} & 0 & y_{\texttt{IE}} z_{\texttt{IE}} \\ \texttt{ID} & 0 & 0 & y_{\texttt{IM}} z_{\texttt{DM}} & 0 & y_{\texttt{IM}} z_{\texttt{DD}} & y_{\texttt{II}} z_{\texttt{DM}} & z_{\texttt{DI}} & y_{\texttt{II}} z_{\texttt{DD}} & 0 & 0 & 0 & y_{\texttt{ID}} & y_{\texttt{IE}} z_{\texttt{DE}} \\ \texttt{Ds} & 0 & 0 & y_{\texttt{DM}} z_{\texttt{SM}} & 0 & y_{\texttt{DM}} z_{\texttt{SD}} & y_{\texttt{DI}} z_{\texttt{SM}} & 0 & y_{\texttt{DI}} z_{\texttt{SD}} & y_{\texttt{DD}} & 0 & 0 & 0 & y_{\texttt{DE}} z_{\texttt{SE}} \\ \texttt{Dm} & 0 & 0 & y_{\texttt{DM}} z_{\texttt{MM}} & 0 & y_{\texttt{DM}} z_{\texttt{MD}} & y_{\texttt{DI}} z_{\texttt{MM}} & 0 & y_{\texttt{DI}} z_{\texttt{MD}} & 0 & y_{\texttt{DD}} & 0 & 0 & y_{\texttt{DE}} z_{\texttt{ME}} \\ \texttt{Di} & 0 & 0 & y_{\texttt{DM}} z_{\texttt{IM}} & 0 & y_{\texttt{DM}} z_{\texttt{ID}} & y_{\texttt{DI}} z_{\texttt{IM}} & 0 & y_{\texttt{DI}} z_{\texttt{ID}} & 0 & 0 & y_{\texttt{DD}} & 0 & y_{\texttt{DE}} z_{\texttt{IE}} \\ \texttt{Dd} & 0 & 0 & y_{\texttt{DM}} z_{\texttt{DM}} & 0 & y_{\texttt{DM}} z_{\texttt{DD}} & y_{\texttt{DI}} z_{\texttt{DM}} & 0 & y_{\texttt{DI}} z_{\texttt{DD}} & 0 & 0 & 0 & y_{\texttt{DD}} & y_{\texttt{DE}} z_{\texttt{DE}} \\ \texttt{EE} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{array} \right)$$

$$\begin{aligned} \Phi &= \{\texttt{SS}, \texttt{sI}, \texttt{MM}, \texttt{mI}, \texttt{MD}, \texttt{IM}, \texttt{iI}, \texttt{ID}, \texttt{Ds}, \texttt{Dm}, \texttt{Di}, \texttt{Dd}, \texttt{EE}\} \\ \mathbf{y} &= \texttt{ConditionalPairMatrix}(\lambda, \mu, r, \ell_1) \\ \mathbf{z} &= \texttt{ConditionalPairMatrix}(\lambda, \mu, r, \ell_2) \\ e_k(\omega, \omega') &= \begin{cases} \exp(\mathbf{Q}(\ell_1 + \ell_2))_{\omega\omega'} & \text{if } k = \texttt{MM} \text{ and } \omega, \omega' \in \Omega \\ \pi_{\omega'} & \text{if } k \in \{\texttt{sI}, \texttt{mI}, \texttt{IM}, \texttt{iI}\},\ \omega = \epsilon, \text{ and } \omega' \in \Omega \\ 1 & \text{if } k \in \{\texttt{MD}, \texttt{Ds}, \texttt{Dm}, \texttt{Di}, \texttt{Dd}\},\ \omega \in \Omega, \text{ and } \omega' = \epsilon \\ 0 & \text{otherwise} \end{cases} \\ \tau(k) &= \begin{cases} \texttt{S} & \text{if } k = \texttt{SS} \\ \texttt{M} & \text{if } k = \texttt{MM} \\ \texttt{I} & \text{if } k \in \{\texttt{sI}, \texttt{mI}, \texttt{IM}, \texttt{iI}\} \\ \texttt{D} & \text{if } k \in \{\texttt{MD}, \texttt{Ds}, \texttt{Dm}, \texttt{Di}, \texttt{Dd}\} \\ \texttt{N} & \text{if } k = \texttt{ID} \\ \texttt{E} & \text{if } k = \texttt{EE} \end{cases} \\ \phi^{(\texttt{S})} &= \texttt{SS} \\ \phi^{(\texttt{E})} &= \texttt{EE} \end{aligned}$$

4.9 Null state elimination

Suppose a model $M = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$ has transition matrix $\mathbf{t}$ and state space Φ. We seek to eliminate all states in $\Psi \subset \Phi$ by incorporating sums over paths through those states into the transition matrix:

$$\texttt{EliminateStates}((\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau); \Psi) = (\Phi, \Omega, \mathbf{t}', \mathbf{e}, \tau)$$

$$\begin{aligned}
\mathbf{t}' &= \mathbf{a} + \mathbf{b}(\mathbf{I} - \mathbf{d})^{-1}\mathbf{c} \\
a_{ij} &= t_{ij}\delta(i \notin \Psi)\delta(j \notin \Psi) \\
b_{ij} &= t_{ij}\delta(i \notin \Psi)\delta(j \in \Psi) \\
c_{ij} &= t_{ij}\delta(i \in \Psi)\delta(j \notin \Psi) \\
d_{ij} &= t_{ij}\delta(i \in \Psi)\delta(j \in \Psi)
\end{aligned}$$

Note that the returned model still has the same state space Φ, but transitions into the eliminated states now have zero weight; this is purely a notational convenience. Implementations can optimize the eliminated states away.

4.10 Stochastic null state restoration

Given a state path ψ through an HMM whose null states Ψ have been eliminated, we restore them by stochastic traceback, as shown in Algorithm 1.

Algorithm 1 $\texttt{RestoreStates}(M, \Psi, \psi)$

Input: $M = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$, $\Psi \subset \Phi$, $\psi \in (\Phi \setminus \Psi)^*$
Output: $\phi \sim \Pr(\phi|\psi)$

```
u ← I + b(I − d)^{−1}                    ▷ b,d defined in Section 4.9
ψ, k ← pop(ψ)                            ▷ k = φ^(E)
φ ← [φ^(E)]
repeat
    ψ, i ← pop(ψ)
    repeat
        For j ∈ Φ: q_j ← u_ij t_jk
        Sample k ∼ Categorical(q/|q|)
        φ ← [k] + φ
    until k = i
until k = φ^(S)
return φ
```

4.11 Forward algorithm

$$\begin{aligned}
\texttt{ForwardMatrix}(X, Y, M) &= F \\
\texttt{ForwardLikelihood}(X, Y, M) &= F\left(L_X, L_Y, \phi^{(\mathtt{E})}\right)
\end{aligned}$$

for Pair HMM $M = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$ and sequences $X, Y \in \Omega^*$.

The Forward matrix F has index space $[0, L_X] \times [0, L_Y] \times \Phi$ and is given by the Forward recursion

$$F(i,j,k) = \begin{cases} E_k(i,j) \sum_{k' \in \Phi} F(i - \Delta_X(k), j - \Delta_Y(k), k') t_{k'k} & \text{if } \tau(k) \neq \mathtt{S} \\ 1 & \text{if } \tau(k) = \mathtt{S} \text{ and } i = j = 0 \\ 0 & \text{otherwise} \end{cases}$$

$$E_k(i,j) = \begin{cases} e_k(X_i, Y_j) & \text{if } \tau(k) = \mathtt{M} \text{ and } i, j > 0 \\ e_k(X_i, \epsilon) & \text{if } \tau(k) = \mathtt{D} \text{ and } i > 0 \\ e_k(\epsilon, Y_j) & \text{if } \tau(k) = \mathtt{I} \text{ and } j > 0 \\ 1 & \text{if } \tau(k) = \mathtt{E},\ i = L_X \text{ and } j = L_Y \\ 0 & \text{otherwise} \end{cases}$$

where E_k is just a notational wrapper to pick out the correct form of the emission probability.

Practically, this should be implemented in log-space to avoid underflow.

4.12 Stochastic Forward traceback

The pseudocode to sample a Pair HMM state path from the posterior distribution given a pair of sequences, by stochastic traceback through the Forward matrix, is given in Algorithm 2.

Algorithm 2 `ForwardTraceback`(M, F)

Input: $M = (\Phi, \Omega, \mathbf{t}, \mathbf{e}, \tau)$, $F \equiv$ `ForwardMatrix`(X, Y, M)
Output: $\phi \sim \Pr(\text{state path} = \phi | X, Y, \Phi, \mathbf{t}, \mathbf{e}, \tau)$

```
(i, j) ← (L_X, L_Y)
k ← φ^(E)
φ ← [φ^(E)]
while k ≠ φ^(S) do
    (i, j) ← (i − Δ_X(k), j − Δ_Y(k))
    For k' ∈ Φ: q_k' ← F(i, j, k') t_k',k
    Sample k' ~ Categorical(q/|q|))
    φ ← [k'] + φ
    k ← k'
return φ
```

4.13 Sampling the intermediate sequence

Let $A, C \in \Omega^*$ be sequences on an endpoint-conditioned trajectory $A \xrightarrow{\ell_1} B \xrightarrow{\ell_2} C$. The pseudocode to generate N *a posteriori* samples of the intermediate sequence B is given in Algorithm 3.

4.13.1 Implementation note: closed-form state elimination

The transition matrices $\mathbf{t}'$ used by `EliminateStates` (Section 4.9), and $\mathbf{u}$ used by `RestoreStates` (Section 4.10), which involve matrix inversions in the general case, can be expressed in closed form for the special case where the model ($M^{(ABC)}$) is the triad HMM and the eliminated state set ($\{\texttt{ID}\}$) consists of that HMM's only null state, as is the case in Algorithm 3. Specifically, if $\mathbf{t}$ is the transition matrix for the triad HMM (Section 4.8), then

$$
\begin{aligned}
t'_{ij} &= \begin{cases} t_{ij} + t_{i,\texttt{ID}} t_{\texttt{ID},j}/(1 - t_{\texttt{ID},\texttt{ID}}) & \text{if } i,j \neq \texttt{ID} \\ 0 & \text{if } i = \texttt{ID} \text{ or } j = \texttt{ID} \end{cases} \\
u_{ij} &= \begin{cases} \delta_{ij} + t_{i,\texttt{ID}}/(1 - t_{\texttt{ID},\texttt{ID}}) & \text{if } i \neq \texttt{ID} \\ \delta_{ij} & \text{if } i = \texttt{ID} \end{cases}
\end{aligned}
$$

4.13.2 Implementation note: caching duplicate samples

As written, Algorithm 3 returns a list of sampled intermediate sequences, together with their posterior probabilities. In practice, for large N and/or small ℓ_1 or ℓ_2, it is likely that some of the sampled sequences will be duplicates. The posterior probabilities for these sequences can be cached; in fact (since the order of the returned B_n is irrelevant) it would be better to return a dictionary mapping each unique B to its corresponding p and the multiplicity with which it was represented in the sampled set.

5 Kullback-Leibler divergence

For given A, t, u we can evaluate the departure from Markovian behavior using a Kullback-Leibler divergence

$$
\mathbb{D}\left[\ M(C|A, t+u)\ ||\ Z(C|A, t, u)\ \right] = \sum_C M(C|A, t+u) \log \frac{M(C|A, t+u)}{Z(C|A, t, u)}
$$

(We could also introduce a prior over A and sum over that, for the full joint distribution over A and C.)

The KL divergence illustrates why it might be useful to be able to compute, or estimate, Z. (This does not address the problem of summing over C, or A if we include that in the KL-divergence; those infinite sums would presumably also need to be approximated by finite samples, e.g. the minibatches from the training data.)

6 Discussion

It is still unclear how to operationalize this as a regularizer term in a training loss. If we simply used minibatches from the training data as an empirical sample from $\Pr(A, C)$ (presumably also using some heuristic to choose t, u), and added the (sample-approximated) KL-divergence

Algorithm 3 `SampleIntermediates`$(A, C, \theta, \ell_1, \ell_2, N)$

Input: $A, C \in \Omega^*$, $\theta = (\lambda, \mu, r, \mathbf{Q}, \pi)$
Output: $B_n \sim \Pr(B|A, C, \theta, \ell_1, \ell_2)$ for $1 \le n \le N$

```
M^(AB) ← ConditionalPairHMM(θ, ℓ1)
M^(BC) ← ConditionalPairHMM(θ, ℓ2)
M^(ABC) ← TriadHMM(θ, ℓ1, ℓ2)
M^(AC) ← EliminateStates(M^(ABC), {ID})                ▷ See §4.13.1
U ← exp(Qℓ1)                                           ▷ Already computed for M^(AB)
V ← exp(Qℓ2)                                           ▷ Already computed for M^(BC)
F^(AC) ← ForwardMatrix(A, C, M^(AC))
p^(AC) ← F^(AC)(L_A, L_C, EE)                          ▷ ForwardLikelihood(A, C, M^(AC))

for n = 1 to N do
    φ^(AC) ← ForwardTraceback(M^(AC), F^(AC))
    φ^(ABC) ← RestoreStates(M^(ABC), {ID}, φ^(AC))     ▷ See §4.13.1
    B_n ← ϵ
    i ← L_A
    j ← L_C
    while φ^(ABC) is nonempty do
        k ← pop(φ^(ABC))
        if k ∈ {MM, IM, MD, ID} then                   ▷ Residue observed at B?
            if k = MM then                             ▷ B aligned to A and C
                For ω ∈ Ω :  q_ω ← U_{A_i,ω} V_{ω,C_j}
            else if k = IM then                        ▷ B aligned to C only
                For ω ∈ Ω :  q_ω ← π_ω V_{ω,C_j}
            else if k = MD then                        ▷ B aligned to A only
                For ω ∈ Ω :  q_ω ← U_{A_i,ω}
            else                                       ▷ B not aligned to A or C
                q ← π
            Sample ω ~ Categorical(q/|q|)
            B_n ← ω B_n
        if k ∈ {MM, MD, Ds, Dm, Di, Dd} then           ▷ Residue observed at A?
            i ← i − 1
        if k ∈ {sI, MM, mI, IM, iI} then               ▷ Residue observed at C?
            j ← j − 1
    p^(AB) ← ForwardLikelihood(A, B, M^(AB))
    p^(BC) ← ForwardLikelihood(B, C, M^(BC))
    p_n ← p^(AB) p^(BC) / p^(AC)                       ▷ p_n = Pr(B_n|A, C, θ, ℓ1, ℓ2)
return [(B_n, p_n) : 1 ≤ n ≤ N]                         ▷ Can cache duplicates; see §4.13.2
```

into the negative log-likelihood (NLL) loss, then it seems to me that the $\log M(C|A, t+u)$ term would cancel out the corresponding NLL term in the loss, and we'd be left minimizing $\log Z$, which seems wrong. Some ideas to fix that:

- Attach a hyperparameter weight $\lambda < 1$ to the KL-divergence penalty, then it won't cancel entirely.
- Make the KL-divergence minibatch different from the NLL minibatch.
- Stop gradients for the $\log M$ term in the KL-divergence (c.f. temporal difference reinforcement learning). This is not exactly addressing the above-identified problem but might help if the loss encourages training to "chase its own tail".
- Abandon KL-divergence and simply add a regularizer that penalizes differences between M and Z in the minibatch; something like $\lambda|\log M(C|A, t+u) - \log Z(C|A, t, u)|$. (The only difference between this and a minibatch-approximation to the KL-divergence is that we're taking the absolute value of the log-odds ratio, so I'm not sure it will do much, but I suppose it might help as the $\log M$ terms no longer have opposing signs in all cases: when $Z > M$ then this regularizer will, like the NLL, be rewarding *increases* in $\log M$.)